■ Design Flash Sale System

System Design Interview | E-Commerce & Marketplace

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Limited inventory sold at discount; Sale starts at exact time; Prevent overselling; Queue users fairly; Order placed on win
Non-Functional	Handle 10-100× normal traffic spike; Sub-second purchase confirmation; No oversell; Fair queuing; Prevent bot abuse
Scale	Xiaomi sold 100K phones in 3 seconds; Amazon Lightning: 10M users competing for 1K items; must handle 1M+ req/sec

■ Architecture for Traffic Spike Absorption

• CDN + static page: pre-render flash sale page; serve from edge; no backend hit until sale starts — absorbs 90%+ traffic

• Virtual waiting room: users enter queue before sale; queue assigns position; prevent thundering herd at T=0

• Rate limiting at API Gateway: per-user token bucket; block bots via CAPTCHA, fingerprinting, purchase history

• Auto-scaling: pre-warm EC2/ECS instances 30 min before sale; scale-out takes 5-10 min — too slow if reactive

■ Inventory Counter (Redis-Based)

• **Atomic Decrement:** Redis DECR inventory:{product_id} — atomic, single-threaded, no race condition; initialize to stock_qty before sale

• **Check & Decrement:** Lua script: if GET > 0 then DECR return 1 else return 0; executes atomically; handles exact last-item race

• **Pre-allocation:** Move stock to Redis before sale; DB updated async after each sale; Redis is source-of-truth during sale window

• **Oversell Guard:** DECR returns new value; if < 0 → INCR to undo and reject; eliminates oversell in Redis layer

■ Virtual Waiting Room / Queue

• On sale open: each user request assigned a queue token (UUID) with timestamp via Redis ZADD queue score=timestamp

• Worker processes queue FIFO: ZPOPMIN to get next user → attempt purchase → notify user via WebSocket/push

• User polling: client polls /queue-status?token=X every 2s to get position and estimated wait time

• Queue size cap: reject new entries after queue length > N×stock_qty; show 'sold out' early to excess users

• Token expiry: if user doesn't complete purchase in 5 min → skip to next in queue; release reservation

■ Bot Prevention & Fairness

• Rate limit: max 1 purchase attempt per user per sale; enforce via user_id + sale_id composite key in Redis

• CAPTCHA: triggered for suspicious patterns (high request rate, datacenter IP, no prior purchase history)
• Purchase history check: new accounts with no history flagged for manual review or limited to 1 item
• Device fingerprinting: detect same device with multiple accounts; browser fingerprint stored in cookie

■■ Queue vs Direct Purchase

Virtual Queue	First-Come-First-Served
✓ Fairer user experience	✓ Simpler implementation
✓ Smooths traffic spike	✓ True speed-based
✓ Complex to implement	✓ Server overwhelmed at T=0
✓ Users wait with feedback	✓ Poor UX for losers

■ Post-Sale Consistency

• After Redis decrements: async Kafka consumer writes each successful purchase to Order DB and Inventory DB
• Reconciliation job: compare Redis final count vs DB orders count; alert if mismatch; idempotent correction
• Analytics: real-time dashboard showing units sold/sec, revenue, conversion rate; Kafka → Flink → dashboard
• Rollback plan: if Redis fails mid-sale → fallback to DB with pessimistic locking (slower but safe)